

VeCroToken: An Efficient, Verifiable, and Privacy-Preserving Cross-Chain Model for Consortium Blockchains Based on zk-SNARKs

Ke Ding[✉], Graduate Student Member, IEEE, Xiaoyan Hu[✉], Member, IEEE, Weicheng Zhou, Guang Cheng[✉], Member, IEEE, Ruidong Li[✉], Senior Member, IEEE, and Hua Wu[✉], Member, IEEE

Abstract—Consortium blockchains enable secure economic applications through privacy-preserving architectures and efficient processing. Growing cross-chain demands require value-exchange mechanisms, yet expose privacy risks during external interactions. Encrypting cross-chain information is necessary, requiring third-party verification of relations within the encrypted content. Existing privacy-preserving cross-chain research for consortium chains struggles to balance transaction efficiency, transaction validity verification, and complex trust assumptions for relays. We present VeCroToken, an efficient, verifiable, and privacy-preserving cross-chain model for consortium blockchains. VeCroToken introduces a dual-balance mechanism and designs four types of cross-chain zero-knowledge transactions based on zk-SNARKs. These transactions encrypt two types of balances and transaction amounts, effectively protecting participant privacy. The encrypted cross-chain data and zero-knowledge proof credentials are stored on participants' consortium blockchains and the relay chain. Relay nodes and third parties can validate transaction proofs with public parameters, preserving privacy while ensuring compliance and validity. We give a security analysis in the UC framework that proves verifiability and balance safety. We also provide a privacy analysis establishing the amount, balance, and fund-correlation privacy. We implement a prototype on Hyperledger Fabric. Our experimental results show that VeCroToken has a lower overall zero-knowledge proof overhead than the state-of-the-art models and performs well in transaction performance.

Index Terms—Consortium blockchain, privacy preservation, verifiability, cross-chain, zero-knowledge proof.

I. INTRODUCTION

IN THE digital age, blockchains offer a decentralized data framework that enables transparent sharing, secure storage, and tamper resistance. However, full transparency on public chains exposes transaction metadata and amounts, which creates privacy risks. In addition, consensus on open public chains, particularly Proof of Work (PoW), leads to high energy consumption and limited throughput. To address these limitations, consortium blockchains (e.g., Hyperledger Fabric [1], [2], Corda [3], [4]) restrict ledger access to authenticated members and provide pluggable and efficient consensus. It improves confirmation latency and operational cost while preserving auditability.

Consortium blockchains are widely used in trade [5], finance [6], and healthcare [7]. As independent networks proliferate, cross-chain information exchange and value transfer become necessary, introducing privacy risks when interacting with external parties [8]. Cross-chain transactions should be encrypted, and third parties should be able to verify required properties, such as verifying the correctness of transactions from ciphertext, without accessing plaintext and avoiding privacy leakage.

Thus, in a consortium blockchain cross-chain scenario, a critical issue is verifying relationships within encrypted private data without compromising data privacy. However, implementing cross-chain models on consortium blockchains faces unique challenges: (1) Consortium chains typically use deterministic consensus protocols (e.g., Practical Byzantine Fault Tolerance (PBFT) [9]), whereas public chains rely on probabilistic mechanisms (e.g., PoW), requiring a redesign of transaction finality guarantees. (2) Permissioned access in consortium chains complicates trust assumptions for cross-chain relay nodes. (3) Performance requirements are stricter in consortium scenarios, demanding lightweight cryptographic primitives. These challenges require immediate attention and practical solutions.

Research on verifiable and privacy-preserving cross-chain consortium blockchain systems is still emerging and warrants further investigation. Specifically, cross-chain privacy designs based on the trusted execution environment (TEE) introduce a dependence on specialized hardware and make auditability rely on operator-maintained logs and attestations. Cross-chain

Received 25 March 2025; revised 23 September 2025 and 15 November 2025; accepted 18 November 2025. Date of publication 21 November 2025; date of current version 13 January 2026. This work was supported in part by National Natural Science Foundation of China Project under Grant 62472087, in part by Jiangsu Province Natural Science Foundation Project under Grant BK20231413, in part by the Fundamental Research Funds for the Central Universities under Grant 2242025K30025, and in part by Postgraduate Research&Practice Innovation Program of Jiangsu Province under Grant KYCX25_0520. The associate editor coordinating the review of this article and approving it for publication was M. Khabbazi. (Corresponding author: Xiaoyan Hu.)

Ke Ding, Weicheng Zhou, Guang Cheng, and Hua Wu are with the School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China (e-mail: keding@seu.edu.cn; wczhou@seu.edu.cn; chengguang@seu.edu.cn; hwu@seu.edu.cn).

Xiaoyan Hu is with the School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China, also with Purple Mountain Laboratories for Network and Communication Security, Nanjing 211111, China, also with the Jiangsu Provincial Engineering Research Center of Security for Ubiquitous Network, Nanjing 211189, China, and also with the Engineering Research Center of Blockchain Application, Supervision and Management, Ministry of Education, Nanjing 211189, China (e-mail: xyhu@njnet.edu.cn).

Ruidong Li is with the Institute of Science and Engineering, Kanazawa University, Kanazawa 920-1192, Japan (e-mail: lrd@se.kanazawa-u.ac.jp).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TNSM.2025.3636014>, provided by the authors.

Digital Object Identifier 10.1109/TNSM.2025.3636014

privacy techniques based on homomorphic encryption [10] and blind signatures [11] reduce cross-ledger transaction linkage, yet privacy could erode under low concurrent transaction volume. By contrast, zk-SNARK-based techniques [12], [13] offer a promising path to privacy and publicly verifiable audit trails. However, existing designs often use overly complex proofs, causing high proving latency for cross-chain transfers and degrading consortium-chain systems' performance.

This paper presents VeCroToken, an efficient, verifiable, and privacy-preserving cross-chain transaction model for consortium blockchains based on zk-SNARKs [14], [15], [16]. VeCroToken enables public verification of statements about hidden data while preserving user privacy and the legal validity of cross-chain transfers. The model introduces a dual-balance mechanism and defines a zero-knowledge token as the medium for cross-chain transactions between consortium blockchains. Each account keeps two balances. One is used locally, and the other is used for cross-chain transfers, which can be verified without exposing amounts. Employing non-interactive zero-knowledge proof technology, it designs and constructs the relevant relational formulas to constrain native account balances, zero-knowledge balances (zk-balances), and transaction amounts, generating a series of parameters. All transaction operations encrypt transaction information and parameters, along with public and private data, and can be used to construct corresponding zero-knowledge proof credentials to verify the relationships among the various private data in transactions. Encrypted transaction data and zero-knowledge proof credentials are recorded on the source consortium chains and on a relay chain, allowing relay nodes and third parties to verify correctness from public data. Our contributions can be summarized as follows:

- Our proposed VeCroToken introduces a dual-balance design and treats zero-knowledge tokens as the medium for cross-chain transfers across consortium blockchains. Using zk-SNARKs, we implement four cross-chain transaction types, including two that encrypt balances and transfer amounts to protect participant privacy. Encrypted transaction data and their proofs are recorded on the source consortium chains and the relay chain, and verifiers use public parameters and data to check correctness, achieving privacy while preserving legal validity.
- We use the preprocessing library jsnark [17] to build zk-SNARK circuits, designing four types of zero-knowledge proof circuits for cross-chain transactions. We also provide a security analysis and a privacy analysis.
- We implement VeCroToken over Hyperledger Fabric, integrating the aforementioned zero-knowledge proof libraries, and deploy a consortium blockchain cross-chain network for evaluating the performance of VeCroToken. The experimental results show that it has a lower overall zero-knowledge proof overhead than the state-of-the-art models and performs well in transaction performance.

The remainder of the paper is organized as follows. We analyze the related work and introduce the background of zk-SNARKs in Section II. Section III presents an overview of the proposed verifiable model, including architectural and transaction data design. Section IV elaborates on the construction of our VeCroToken model and analyzes the security of the

circuit construction. Section V evaluates the performance of VeCroToken and compares it with other methods. Section VI discusses potential directions for optimizing VeCroToken. Finally, we conclude this paper in Section VII.

II. RELATED WORK & BACKGROUND

This section briefly reviews privacy preservation and information verifiability on blockchains, cross-chain technology, privacy-preserving cross-chain transactions, and the relevant background on zk-SNARKs.

A. Privacy Preservation and Information Verifiability on Blockchains

Privacy protection in blockchains must simultaneously ensure the confidentiality of on-chain values and the public verifiability of legal correctness. Early techniques, such as coin mixing and ring signatures, mainly target public Unspent Transaction Output (UTXO) systems and are difficult to port to consortium blockchains that require fine-grained auditability by third parties and deterministic finality. Many consortium platforms adopt account-based or key-value world states with private channels and identity-based endorsement policies rather than a native global set of unspent outputs, which makes direct reuse of UTXO-oriented techniques nontrivial.

In contrast, zero-knowledge proofs provide confidentiality while enabling public verification of statements about hidden data, better matching the requirements of consortium settings that need accountable validation across organizational boundaries. Succinct non-interactive arguments of knowledge (zk-SNARKs) [14], [18] offer short proofs and fast verification, and are widely used to protect transaction privacy while certifying conservation and range relations of hidden values [19]. Monero mainly uses Bulletproofs [20] to reduce proof sizes in confidential transactions further, though they typically trade verification costs against succinctness. Account-model privacy systems [21] demonstrate that zero-knowledge can be integrated without relying on UTXO abstractions. Building upon this line, our previous VmppToken [22] focuses on single-chain consortium transactions where privacy and third-party verifiability are required. The cross-chain setting introduces additional challenges, including consistency across ledgers and freshness guarantees for inter-chain transfers. These motivate our model's design choices to leverage zk-SNARKs to attest balance relations and uniqueness under encrypted transcripts, enabling privacy preservation with auditable verification suitable for consortium blockchains.

B. Cross-Chain Technology and Privacy-Preserving Cross-Chain Methods

Cross-chain interaction has evolved along several architectural lines, including notary-based protocols that rely on intermediaries, sidechains that extend main-chain functionality, relay-chain designs that record cross-chain evidence, and hash-locked swaps for atomic exchange. Industrial systems exemplify these approaches at scale [23], [24], [25]. Wanchain [26] employs a decentralized group of cross-chain

nodes to execute value transfer and message passing across heterogeneous ledgers with threshold-managed custody and routing, aiming at compatibility with major public-chain stacks. Harmony's bridge [27] connects EVM-compatible domains and illustrates the operational considerations of externally validated bridges, including the trust placed in relays and the need for robust monitoring and revocation mechanisms. Chainlink CCIP [28] offers a cross-chain interoperability protocol powered by decentralized oracle networks. It introduced private transactions [29] that encrypt cross-chain messages and transaction parameters. Institutions can use the public CCIP network to connect private chains while keeping sensitive data confidential and compliant. These production systems enable interoperability but generally do not protect the amounts or balances involved in transfers by default. Even when confidentiality is provided through payload encryption, publicly auditable zero-knowledge proofs of value conservation remain uncommon, limiting third-party verification without plaintext access.

The research community has explored some privacy-preserving cross-chain techniques. Li et al. [30] proposed an interoperability framework that runs cross-chain smart contracts on paired TEE hosts (execution/verification), preserving I/O confidentiality and ensuring low-cost, race-free operation via incentives and non-blocking optimistic concurrency control. Belchior et al. [31] proposed a TEE-based fault-tolerant middleware that extends the open digital asset protocol with a gateway two-phase commit and tamper-evident logs for cross-ledger transfers. TEE-based cross-chain privacy schemes rely on specialized hardware roots of trust, and their audits hinge on operator logs and attestations, raising concerns about availability, integrity, completeness, and equitable access.

Cai et al. [10] replaced the hashed timelock contract's shared hashlock with Paillier homomorphic timelock parameters agreed off-chain, yielding distinct on-chain locks across ledgers to prevent parameter-based cross-chain transaction linkage while retaining atomic settlement. Hanzlik et al. [11] used blind signatures with an intermediary sweeper, timelocks, and anonymous channels so that each leg appears as a standard transfer, achieving cross-chain transaction unlinkability under concurrent swaps and preserving fungibility. These two approaches emphasize linkability reduction but leave on-chain metadata (amounts, addresses, timestamps) in the clear, permitting inference attacks under limited concurrency.

Guo et al. [12] proposed the zkCross, a two-layer (Lower Layer/Audit Layer) cross-chain architecture combining zk-SNARK-based private transfer and exchange protocols with a succinct auditing protocol. It enables privacy-preserving and efficient cross-chain auditing. Liu et al. [13] proposed a hierarchical privacy-preserving verifiable blockchain system, HyperMaze, which essentially follows a relay-chain-based cross-chain approach. By adopting zk-SNARKs technology, it can effectively ensure privacy preservation and verifiability of transactions. However, these zk-SNARK-based methods often incur excessive zero-knowledge proof overhead for cross-chain transfers, significantly burdening the performance of consortium-chain systems.

C. zk-SNARKs

Since the 1980s, zero-knowledge proof has evolved into a critical technology in the field of privacy preservation. The concept, first introduced by Goldwasser et al. [32], enables a prover to convince a verifier of the truth of a statement without revealing any information beyond the validity of the claim. However, the original concept of zero-knowledge proof was interactive, meaning that the prover and verifier had to engage in repeated interactions to convince the verifier that the prover knew the correct information. This interactive protocol was time-consuming and resource-intensive, and its drawbacks became particularly evident when involving large amounts of data.

To address these issues, Blum et al. [33] introduced the concept of non-interactive zero-knowledge proofs in 1988. Their research showed that introducing a common reference string (CRS) could achieve zero knowledge without any interaction. In 2012, Bitansky et al. [34] proposed zero-knowledge succinct non-interactive arguments of knowledge (zk-SNARKs), extending the application scope of zero-knowledge proof. This approach maintained zero knowledge while significantly reducing the proof size and verification time. Zk-SNARKs are among the most efficient and widely used zero-knowledge proof schemes.

Zk-SNARKs can be represented by a quadruple of probabilistic polynomial-time (PPT) algorithms $\Pi = (\text{Setup}, \text{GenKey}, \text{Prove}, \text{Verify})$ such that:

- $\text{Setup}(\lambda) \rightarrow \xi$: Given a security parameter λ , the setup produces a set of basic common parameters ξ on the bilinear group. These parameters are reused across all circuits and algorithms.
- $\text{GenKey}(C) \rightarrow (pk, vk)$: Given a circuit C (and ξ), run a trusted setup to derive a circuit-specific common reference string (CRS) σ and the key pair (pk, vk) , where pk (proving key) is used to generate zk-SNARK proofs and vk (verifying key) is used for public verification.
- $\text{Prove}(pk, \vec{x}, \vec{w}) \rightarrow \pi$: Using pk and witness $\vec{w}$, output a proof π attesting that there exists $\vec{w}$ such that $C(\vec{x}, \vec{w}) = 1$. Here $\vec{x}$ is public, while $\vec{w}$ remains private. In our paper, π certifies the correctness of a balance update or a transaction while the underlying values remain hidden.
- $\text{Verify}(vk, \vec{x}, \pi) \rightarrow \{0/1\}$: Using vk , return 1 iff π is a valid proof that $C(\vec{x}, \vec{w}) = 1$ for some (hidden) $\vec{w}$, otherwise return 0.

The pair (pk, vk) is bound to the circuit C . As long as C does not change, the same keys can be reused to generate and verify arbitrarily many proofs, and previously issued proofs remain valid under their corresponding vk . Any change to C , or any compromise of the trusted setup, requires running GenKey again to obtain fresh keys.

zk-SNARKs satisfy three core properties of zero-knowledge proofs: Completeness, Soundness, and Zero-knowledge [35].

Completeness: If the prover's statement is true and both the prover and verifier honestly follow the protocol's steps, then the verifier must accept the statement, that is, for all

$(\vec{x}, \vec{w}) \in C$,

$$\Pr \left[\text{Verify}(vk, \vec{x}, \pi) = 1 \mid \begin{array}{l} \text{GenKey}(C) \rightarrow (pk, vk) \\ \text{Prove}(pk, \vec{x}, \vec{w}) \rightarrow \pi \end{array} \right] = 1.$$

Soundness: The verifier will accept the statement only when the prover's claim is valid. It implies that the probability of a PPT adversary $\mathcal{A}$ successfully proving a false statement is 0:

$$\Pr \left[\begin{array}{l} (\vec{x}, \vec{w}) \notin C \\ \text{Verify}(vk, \vec{x}, \pi) = 1 \end{array} \mid \begin{array}{l} \text{GenKey}(C) \rightarrow (pk, vk) \\ \mathcal{A}(C, pk, vk) \rightarrow (\vec{x}, \vec{w}, \pi) \end{array} \right] = 0.$$

Zero-knowledge: The verifier should not learn any information other than the fact that the statement is true. This property is achieved by constructing a polynomial-time simulator $Sim = (S_1, S_2)$, where S_1 returns parameters pk , vk , and a simulation trapdoor τ , enabling S_2 to simulate a credential π without private data $\vec{w}$, that is, for all probabilistic polynomial adversaries $\mathcal{A}$,

$$\Pr \left[\mathcal{A}(pk, vk, \pi) = 1 \mid \begin{array}{l} \text{GenKey}(C) \rightarrow (pk, vk) \\ \text{Prove}(pk, \vec{x}, \vec{w}) \rightarrow \pi \end{array} \right] \approx \Pr \left[\mathcal{A}(pk, vk, \pi) = 1 \mid \begin{array}{l} S_1(C) \rightarrow (pk, vk, \tau) \\ S_2(pk, \vec{x}, \tau) \rightarrow \pi \end{array} \right].$$

III. MODEL OVERVIEW

This section overviews our proposed VeCroToken, a verifiable and privacy-preserving cross-chain model for consortium blockchains based on zk-SNARKs. We describe its architecture design and present the data involved in the model.

A. Architecture Design

VeCroToken adopts a dual-balance design: a native balance for ordinary on-chain accounting and a zk-balance for privacy-preserving cross-chain transfers. Users can pre-convert part of their native tokens into zk-tokens and then perform multiple cross-chain transfers without reconverting each time. Users interact through four transaction operations: Deposit, Withdraw, Send, and Receive. Each operation commits encrypted values (balances and amounts) to the relevant chains, together with a zk-SNARK proof. A relay chain verifies the proof using public parameters and records the verified outcome for third-party auditing, providing public verifiability and compliance transparency without revealing sensitive values.

Fig. 1 shows the model design. Initially, VeCroToken performs parameter initialization and generates zero-knowledge proving and verifying key pairs for the four transaction circuits (Deposit, Withdraw, Send, and Receive). A user then initializes an account with a name, a public key, and encrypted balances, while the private key remains off-chain. To enable third-party validation without access to plaintext balances, the relay chain retains the identifiers, statuses, and proofs required for verifying all four transaction types.

Deposit/Withdraw transactions convert value between the native balance and the zk-balance via the public conversion factor. The relay chain verifies the proofs with the verifying keys for Deposit and Withdraw, checks for freshness via the unique transaction identifier, updates the transaction status, and

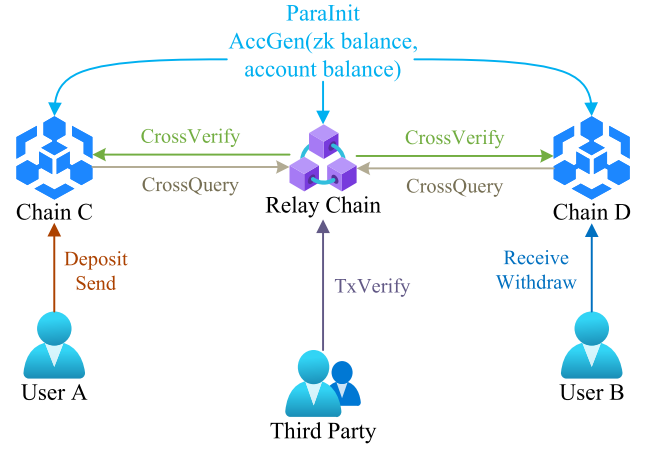


Fig. 1. Architecture design of VeCroToken.

records the verified outcome, exposing only ciphertexts and proofs.

Transfer transactions are divided into two parts: Send and Receive transactions. The sending party initiates the Send transaction and generates a zero-knowledge proof credential to prove the consistency of zk-balances before and after the transaction, and the non-negativity of the zk-balance and the transaction amount after the transaction. The transfer amount is encrypted under a symmetric key, while the symmetric key is asymmetrically encrypted under the recipient's public key. After the Send information is posted and verified on the relay chain, the receiving party locates this transfer by referencing the send transaction identifier with a receive transaction identifier, decrypts the transfer amount, and initiates the Receive transaction with a receiver-side proof that credits the receiver's zk-balance by the same amount. The relay chain verifies pair consistency with the verifying key for Receive, closes the pending entry by marking the corresponding Send as finished, and updates the statuses accordingly.

As motivating examples, consider two real-world regulated deployments. In cross-border interbank transfers between consortium blockchains C and D , the sending bank locks value on C and posts an encrypted transfer commitment to D . A relay records a bundle of zero-knowledge proofs showing that sufficient value is reserved, the transfer is unique, and release conditions on D are satisfied. Meanwhile, amounts and balances remain hidden from uninvolved members and the relay itself. Regulators can later verify the proofs' validity without plaintext. In supply-chain finance, a manufacturer/logistics consortium and a finance consortium remain segregated for confidentiality, but an approved receivable must move across the chains for early funding. The supplier conveys an encrypted claim representing the receivable across chains, accompanied by proofs that it is eligible, correctly accounted for, and used once only. Commercial terms (prices, discounts, and credit limits) stay confidential to counterparties, while the relay maintains public verifiability by recording proof artifacts rather than trade data. These deployments require that privacy and public verifiability must coexist across consortium chains.

TABLE I
DATA DESIGN OF VECROTOKEN

Data Content	Label	Used by
Account Name	$N; N_A; N_B$	Account / Deposit / Withdraw / Send / Receive
Account Public Key	$pubk; pubk_A; pubk_B$	Account / Deposit / Withdraw / Send / Receive
Transaction Identifier	$Tid; Tid_A; Tid_B$	Deposit / Withdraw / Send / Receive
Encrypted Account Token Value (pre/post)	$E_v, E_v^*; E_{vA}, E_{vA}^*; E_{vB}, E_{vB}^*$	Account / Deposit / Withdraw
Encrypted Zk-Token Value (pre/post)	$E_z, E_z^*; E_{zA}, E_{zA}^*; E_{zB}, E_{zB}^*$	Account / Deposit / Withdraw / Send / Receive
Token Conversion Ratio Factor	n_c	Deposit / Withdraw
Encrypted Transaction Amount	M	Send / Receive
Encrypted Symmetric Key (under receiver's public key)	E_k	Send
Zero-knowledge Proof Credential	π	Deposit / Withdraw / Send / Receive
Transaction Status	sta	Deposit / Withdraw / Send / Receive

¹ A denotes the sender. B denotes the receiver. $*$ denotes the post-transaction state.

B. Transaction Data Design

Transaction data can be divided into Account, Deposit/Withdraw, Send, and Receive. The data design of these transactions is shown in Table I.

Account data on two ledgers. We create a consortium-chain account and a relay-chain account for the same user: $Acc_N = \{N, E_v, E_z, pubk\}$ (on the consortium chain) and $RAcc_N = \{N, E_z, pubk\}$ (on the relay chain). Here E_v and E_z are encryptions of the native balance v and the zk-balance z under $pubk$, while the private key $prik$ never leaves the client. $RAcc_N$ provides a minimal view to support cross-chain recording and counterparty queries during transfers.

Deposit/Withdraw transaction data. Each transaction carries an identifier Tid and a status sta . A public coefficient n_c , set and updated through governance, regulates the conversion between native and zk-tokens. The actual native balance values before and after the transaction are v and v^* , while the actual zk-balance values before and after the transaction are z and z^* . The actual transaction amount is m . All of v, v^*, z, z^* , and m are private data not posted on-chain. The posted data include the pre-/post-transaction-ciphertexts (E_v, E_v^*) and (E_z, E_z^*) and a proof π asserting conservation and non-negativity under n_c . After the consortium chain records the transaction, the relay chain verifies π with the corresponding verifying key, checks freshness via Tid , updates sta , records the verified transaction, and writes the updated $RAcc_N$ to the relay ledger for later auditing.

Send transaction data. Send transaction carries Tid_A , sender name N_A , receiver name N_B , public key $pubk_A$, pre-/post-transaction-ciphertexts of zk-token values (E_{zA}, E_{zA}^*) , the encrypted amount M , an encrypted symmetric key (under the receiver's public key) E_k , a send zero-knowledge proof credential π , and a status sta . Given the actual transaction amount m and the symmetric key k , the encrypted transaction amount is $M = E(m, k)$. To ensure the privacy of k , it must be encrypted with the receiver's public key, resulting in the encrypted symmetric key E_k . π proves the debit $E_{zA} \rightarrow E_{zA}^*$ by amount M with non-negativity and freshness. Upon relay verification, the transaction status sta is marked pending on the relay chain until the matching Receive completes.

Receive transaction data. The receiver references the sender's Tid_A with its own Tid_B and account name N_B , and submits π together with $pubk_B$ and the pre-/post-ciphertexts

TABLE II
OPERATION DATA RELATIONSHIPS

Operation	$z^* - z$	$v^* - v$	Guard(s)
Deposit	$+m$	$-n_c m$	$m > 0, v^* \geq 0$
Withdraw	$-m$	$+n_c m$	$m > 0, z^* \geq 0$
Send	$-m$ (sender z_A)	-	$m > 0, z_A^* \geq 0$
Receive	$+m$ (receiver z_B)	-	pair with Send

(E_{zB}, E_{zB}^*) , proving a matching credit by the same M . The relay chain validates π , enforces pair consistency with the referenced Send (including equality of M), sets the Send status to Finished, updates sta for Receive, and writes the updated $RAcc_N$ for the receiver to the relay ledger. The consortium chain then updates the receiver's Acc_N with E_{zB}^* .

IV. VECROTOKEN MODEL

This section presents the model design of VeCroToken. We first introduce its construction in detail and then analyze its security and privacy.

A. Construction of VeCroToken

VeCroToken consists of a set of polynomial-time algorithms $\Psi = (\text{Paralnit}, \text{AccGen}, \text{Deposit}, \text{Withdraw}, \text{Send}, \text{Receive}, \text{CrossVerify}, \text{TxVerify})$.

Specifically, Table II presents the transaction data relations for the four transaction operations. For all operations, native-balance ciphertexts satisfy $E_v = EA(v, pubk)$ and $E_v^* = EA(v^*, pubk)$. The zk-balance ciphertexts are bound to their plaintexts under the account public key: $E_z = EA(z, pubk)$ and $E_z^* = EA(z^*, pubk)$. For *Send/Receive*, the amount payload satisfies $M = E(m, k)$ and $E_k = EA(k, pubk_B)$. Each transaction binds a unique Tid for freshness and replay protection. VeCroToken enforces these relationships and constraints through zero-knowledge proofs.

1) $\text{Paralnit}(\lambda, C_i) \rightarrow (pk_i, vk_i), i \in \{dep, wd, send, rece\}$. Given a security parameter λ and four operation circuits $C_{dep}, C_{wd}, C_{send}, C_{rece}$, the algorithm generates some basic parameters ξ and four key pairs, $(pk_{dep}, vk_{dep}), (pk_{wd}, vk_{wd}), (pk_{send}, vk_{send})$, and (pk_{rece}, vk_{rece}) . pk_i is used to generate the zero-knowledge proof credential, and vk_i is used to verify the credential. Without changing C_i , the algorithm only needs to be executed once, and the consortium blockchain can

perform the initial parameter generation. Hence, the key pair is reusable. If a circuit is modified (feature change or bug fix), we rerun GenKey to derive fresh (pk, vk) . Supplementary material Listing 1 specifies Paralnit as the one-time off-chain setup run by consortium operators to derive and publish the proving and verification keys for all four transaction circuits before any VeCroToken transaction is executed.

2) $\text{AccGen}(N, C) \rightarrow \{Acc_N, RAcc_N, prik\}$. Given an account name N on consortium blockchain C , the algorithm creates a consortium-chain account and a relay-chain account. It first generates an RSA key pair $(pubk, prik)$, initializes $v = z = 0$, and computes $E_v = EA(v, pubk)$ and $E_z = EA(z, pubk)$. It then constructs $Acc_N = \{N, E_v, E_z, pubk\}$ and the corresponding relay record $RAcc_N = \{N, E_z, pubk\}$ for cross-chain recording and querying. Finally, the user is privately informed of $prik$, and Acc_N and $RAcc_N$ are written to ledger L_C and relay ledger L_R , respectively. Supplementary material Listing 2 formalizes AccGen as a client-side procedure that generates the RSA key pair, initializes the encrypted balances, and writes synchronized account records to both the consortium ledger and the relay ledger.

3) $\text{Deposit}(N, C, m, prik, pk_{dep}) \rightarrow \{Acc_N, Tx_{dep}, v^*, z^*\}$. This algorithm enables account N to perform a Deposit on consortium blockchain C with amount m using $prik$, converting part of the native balance into the zk-balance. It first checks $m > 0$, then retrieves Acc_N from L_C by N to obtain $(pubk, E_v, E_z)$. Using $prik$, it decrypts v and z , and computes z^* and v^* from m and the chain's conversion coefficient n_c . It enforces $v^* \geq 0$ (else fail), derives E_z^* and E_v^* , and runs *Prove* with pk_{dep} over the public/secret inputs to generate the credential π_{dep} attesting the range/equality relations in Table II. Supplementary material Listing 3 instantiates Deposit as the procedure run by the user client on consortium chain C together with the relay, from local balance checks and construction of public and secret inputs to the call to CrossVerify.

The algorithm then assembles Tx_{dep} (including π_{dep} and required public data) and appends it to the blockchain with $sta = \text{"Valid"}$. It invokes CrossVerify, and the relay returns $r \in \{0, 1\}$. If $r = 0$, the transaction is recorded with $sta = \text{"Invalid"}$ (non-usable). If $r = 1$, it is recorded with $sta = \text{"Deposit"}$ to mark completion, after which third parties can verify its correctness. Finally, Acc_N is updated on L_C , and the user is informed of the new balances v^* and z^* . Fig. 2 shows the construction of the zero-knowledge proof circuit for the Deposit operation, where the private data are $\{z, z^*, m, v, v^*\}$ and the rest are publicly available data.

4) $\text{Withdraw}(N, D, m, prik, pk_{wd}) \rightarrow \{Acc_N, Tx_{wd}, v^*, z^*\}$. This algorithm enables account N to perform a Withdraw on consortium blockchain D with amount m using $prik$, converting zk-balance into native balance. It first checks $m > 0$, then retrieves Acc_N from L_D to obtain $(pubk, E_v, E_z)$, and using $prik$, decrypts v and z . It then computes v^* and z^* , enforces $z^* \geq 0$ (else fail), derives E_v^* and E_z^* , and runs *Prove* with pk_{wd} over the public/secret inputs to generate the credential π_{wd} attesting the relations in Table II. After assembling Tx_{wd} (including π_{wd} and required public data), the transaction is added to the blockchain. The relay chain

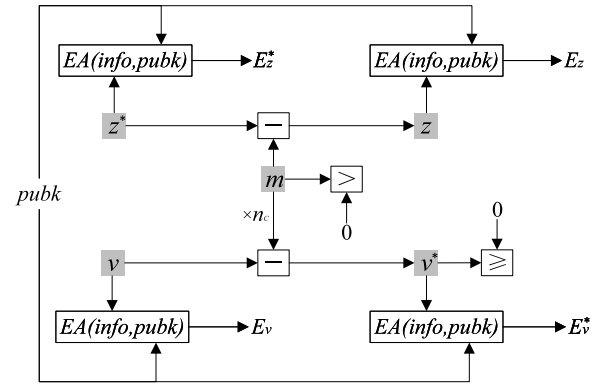


Fig. 2. The construction of the zero-knowledge circuit for the Deposit operation.

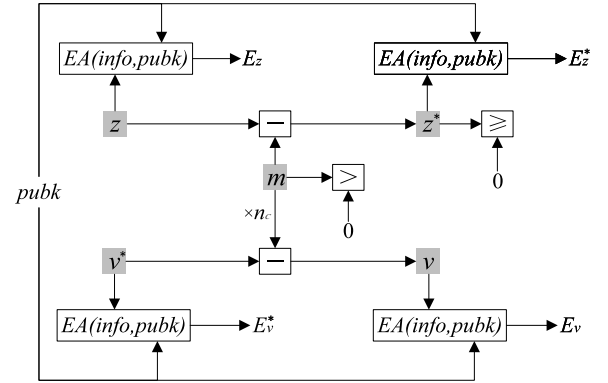


Fig. 3. The construction of the zero-knowledge circuit for the Withdraw operation.

verifies legality and validity and updates the status accordingly. Upon completion, Acc_N is updated on L_D , and the user is informed of the post-transaction balances v^* and z^* .

Fig. 3 shows the construction of the zero-knowledge proof circuit for the Withdraw operation, where the private data are $\{z, z^*, m, v, v^*\}$, and the rest are publicly available data.

5) $\text{Send}(N_A, C, prik_A, N_B, m, k, pk_{send}) \rightarrow \{Acc_{NA}, Tx_{send}, z_A^*\}$. This algorithm enables account N_A on consortium chain C to initiate a cross-chain *Send* to N_B using $prik_A$, amount m , and symmetric key k . It checks $m > 0$, retrieves Acc_{NA} from L_C to obtain $(pubk_A, E_z)$, decrypts z_A with $prik_A$, computes $z_A^* = z_A - m$, enforces $z_A^* \geq 0$, and derives $E_{z_A}^*$ under $pubk_A$. To avoid exposing the receiver's private key in circuits, the amount is encrypted symmetrically as $M = E(m, k)$, and the key is encapsulated as $E_k = EA(k, pubk_B)$. It then runs *Prove* with pk_{send} over the public/secret inputs to generate the credential π_{send} attesting the relations in Table II. The transaction Tx_{send} , including π_{send} and required public data (e.g., E_k), is posted with $sta = \text{"Valid"}$ pending relay verification.

After CrossVerify returns $r \in \{0, 1\}$, if $r = 0$ the transaction is re-posted with $sta = \text{"Invalid"}$. If $r = 1$ it is re-posted with $sta = \text{"Send"}$ to indicate sender-side completion. Note that this status alone does not imply end-to-end completion. The transfer is finalized only when the relay records the matched *Send/Receive*. Finally, Acc_{NA} is updated on L_C , and the user

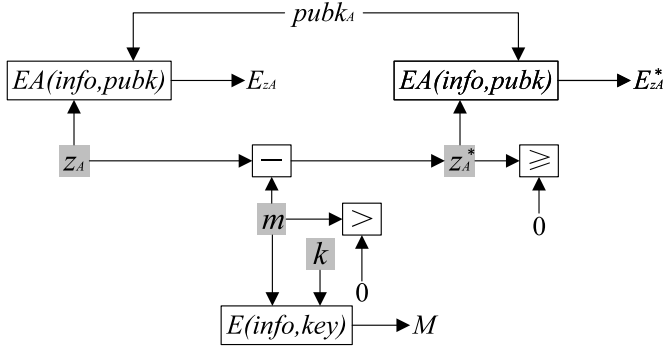


Fig. 4. The construction of the zero-knowledge circuit for the Send operation.

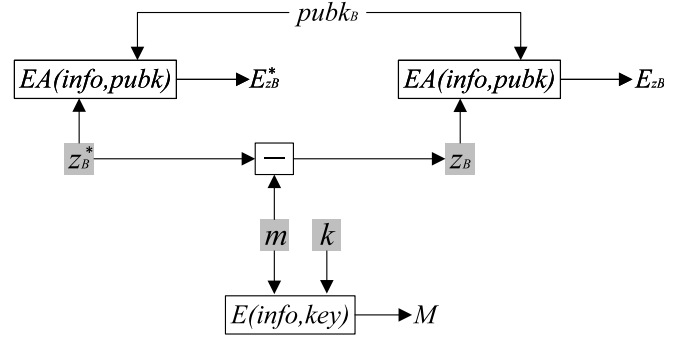


Fig. 5. The construction of the zero-knowledge circuit for the Receive operation.

is informed of the updated zk-balance z_A^* . Supplementary material Listing 4 details the Send procedure executed by the sender on consortium chain C, including derivation of the encrypted amount and encapsulated key, generation of the proof π_{send} and posting of the transaction to CrossVerify on the relay chain.

Fig. 4 shows the construction of the zero-knowledge proof circuit for the Send operation, where the private data are $\{z_A, z_A^*, m, k\}$, and the rest are publicly available data.

6) $\text{Receive}(N_B, D, \text{pri}k_B, \text{pk}_{\text{rece}}) \rightarrow \{Acc_{NB}, Tx_{\text{rece}}, z_B^*\}$. This algorithm enables account N_B on destination consortium chain D to complete a Receive corresponding to a pending Send. It locates Tx_{send} on the relay ledger L_R by N_B and checks $sta = \text{"Send"}$. From Tx_{send} it obtains (E_k, M) , using $\text{pri}k_B$ it recovers k from E_k and then m from $M = E(m, k)$. It retrieves $(\text{pub}k_B, E_{zB})$ from L_D , decrypts z_B with $\text{pri}k_B$, computes $z_B^* = z_B + m$, and derives E_{zB}^* under $\text{pub}k_B$. It runs *Prove* with pk_{rece} over the public/secret inputs to generate π_{rece} attesting the relations in Table II. Then it forms Tx_{rece} (including Tid_A and required public data) and posts it with $sta = \text{"Valid"}$ for relay verification.

After CrossVerify returns $r \in \{0, 1\}$, if $r = 0$ the transaction is re-posted with $sta = \text{"Invalid"}$. If $r = 1$ it is re-posted with $sta = \text{"Receive"}$, paired to the originating Send. Finally, Acc_{NB} is updated on L_D , and the user is informed of the updated zk-balance z_B^* . Supplementary material Listing 5 describes the Receive procedure executed by the receiver on the destination consortium chain, locating the pending Send on the relay ledger, decrypting the ciphertexts and constructing the proof π_{rece} before invoking CrossVerify.

Fig. 5 shows the construction of the zero-knowledge proof circuit for the Receive operation, where the private data are $\{z_B, z_B^*, m, k\}$, and the rest are publicly available data.

7) $\text{CrossVerify}(N, C, Tid, vk_{\text{dep}}, vk_{\text{wd}}, vk_{\text{send}}, vk_{\text{rece}}) \rightarrow \{RAcc_N, Tx, r\}$. This algorithm performs relay-chain validation for *Deposit*, *Withdraw*, *Send*, and *Receive*. Given (C, Tid) , it fetches Tx from L_t^C and requires $sta = \text{"Valid"}$. If $sta = \text{"Invalid"}$, the transaction previously failed relay verification. If $sta \in \{\text{"Deposit"}, \text{"Withdraw"}, \text{"Send"}, \text{"Receive"}\}$, it is finalized and non-reusable, preventing reuse or double-spend. The algorithm then identifies the transaction type, parses the public transcript, and runs $\text{Verify}(vk_{\text{type}}, \cdot)$ on the credential π . Supplementary material Listing 6 presents the

CrossVerify procedure on the relay ledger, which branches on the four transaction types, verifies the zk-SNARK proof, and updates both the transaction status and the relay account record $RAcc_N$. For a *Receive* transaction, the relay locates the paired Tx_{send} via Tid_A on the relay ledger, checks that the public messages match $M_{\text{rece}} = M_{\text{send}}$, and verifies that the sender-side status satisfies $sta(Tx_{\text{send}}) = \text{"Send"}$, thereby ensuring that each *Send* is consumed at most once. It also enforces receiver identity binding by requiring that the account ciphertext E_{zB} in Tx_{rece} equals the receiver's account certificate E_{zB}^{ledger} on the destination ledger. Let $r \in \{0, 1\}$ be the verification result. If $r = 0$, the relay marks the paired Tx_{send} (for *Receive*) as "Invalid" and returns r . If $r = 1$, it writes back the updated Tx and $RAcc_N$ to the relay ledger L_R , marks the consumed identifier as "Finished" to prevent future reuse, and then returns r .

8) $\text{TxVerify}(Tid, vk_{\text{dep}}, vk_{\text{wd}}, vk_{\text{send}}, vk_{\text{rece}}) \rightarrow r$. This algorithm enables third-party verification of *Deposit*, *Withdraw*, *Send*, and *Receive* using only the relay ledger. Given Tid , it queries L_R for Tx , confirms its status, and determines the transaction type. If Tx is *Deposit/Withdraw*, it extracts $(\text{pub}k, E_v, E_v^*, E_z, E_z^*, \pi_{\text{dep/wd}})$ and runs $\text{Verify}(vk_{\text{dep/wd}}, \cdot)$ on the credential. If Tx is *Send*, it extracts the public transcript and π_{send} and verifies with vk_{send} . If Tx is *Receive*, it also reads Tid_A from Tx , queries L_R for Tx_{send} , verifies π_{send} with vk_{send} , and requires M equality between Tx_{send} and Tx_{rece} as well as $sta(Tx_{\text{send}}) = \text{"Finished"}$. It then verifies π_{rece} with vk_{rece} and returns r . For *Receive*, $r = 1$ only if both π_{send} and π_{rece} are valid and the matching conditions hold, otherwise $r = 0$.

B. Security Analysis

In VeCroToken, both types of balances (account balance and zk-balance) and transaction amounts are encrypted for protection. Moreover, the relay chain and third parties can verify the legality and effectiveness of the four types of transactions using public data, ensuring that the private data in transactions adhere to correct relationships. The zero-knowledge proof circuits for Deposit transactions guarantee that VeCroToken is immune to overspending attacks on account balances, while those for Withdraw and Send transactions ensure immunity to overspending attacks on zk-balances. Additionally, by marking transaction information statuses on the consortium

blockchain and relay chain, VeCroToken is safeguarded against double-spending attacks. This section defines the security of VeCroToken under the Universal Composability (UC) framework [36]. The supplementary material [37] provides a detailed UC model of VeCroToken, including the ideal functionalities and the simulator used in our security and privacy analyses.

1) *Threat and Adversary Model*: We consider a cross-ledger setting with two consortium chains C, D connected via a relay chain R . Parties include transaction senders/receivers, validators on each consortium chain, and relay validators. We adopt a static corruption model: the adversary may corrupt a minority of validators on any chain, but consensus safety holds under the assumed Byzantine fault tolerance (e.g., PBFT-type majority of honest stake/nodes). The adversary is a PPT algorithm with full network scheduling power. It may attempt (i) to forge cross-chain proofs; (ii) to replay or concurrently submit states so as to trigger double-spending or overspending; and (iii) to cause inconsistent state transitions across chains. All cryptographic primitives meet standard assumptions: signatures are EUF-CMA [38] secure (i.e., existentially unforgeable under adaptive chosen-message attacks); commitments are computationally binding and hiding; symmetric encryption and public-key encryption satisfy unique-decodability; and zk-SNARKs satisfy completeness, knowledge soundness, and zero-knowledge. Public parameters are assumed to be generated by a trusted setup for the underlying proving system. Concretely, we assume a multi-party computation (MPC) ceremony with at least one honest participant, with ceremony transcripts and randomness-beacon sources published.

2) *UC Modeling Overview*: We analyze security in a UC-inspired setting with a relay-mediated cross-ledger protocol Π . Adversary $\mathcal{A}$ is PPT with full network scheduling power; the environment $\mathcal{Z}$ supplies inputs and reads outputs. We work in a $(\mathcal{G}_{\text{Ledger}}, \mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{XVer}})$ -hybrid model [37]. $\mathcal{G}_{\text{Ledger}}$ abstracts the consortium/relay ledgers with *deterministic finality* and BFT [9] safety/liveness. $\mathcal{F}_{\text{CRS}}$ provides public parameters for a zk-SNARK with completeness and soundness. $\mathcal{F}_{\text{XVer}}$ provides the public relay-side verification oracle used by CrossVerify.

3) *Verifiability & Balance Safety*: The ideal functionality $\mathcal{F}_{\text{VCT}^*}$ ensures the correctness of the balances and the validity of operations. It enforces the validity of operations by ensuring they satisfy the relevant balance update constraints. The state of each account is updated based on the specific conditions of each operation.

State. Per account: $(\text{Bal}_{\text{acc}}, \text{Bal}_{\text{zk}})$; per transfer id Tid : status $\text{sta} \in \{\text{Send}, \text{Finished}, \text{Invalid}\}$.

Interfaces. Four calls enforce relations R_{Op} and update state. Only public transcripts (status and public inputs) are emitted to $\mathcal{G}_{\text{Ledger}}$.

- *Deposit*(a, m): require $m > 0$ and $z^* - z = m$, $v - v^* = n_c m$, $v^* \geq 0$. Update balances and publish success/failure.
- *Withdraw*(a, m): require $m > 0$ and $z - z^* = m$, $v^* - v = n_c m$, $z^* \geq 0$. Update balances and publish success/failure.

- *Send*($a \rightarrow b, m, \text{Tid}$): require $m > 0$ and $z_{\text{snd}}^* = z_{\text{snd}} - m$, $z_{\text{snd}}^* \geq 0$. Register Tid with $\text{sta} = \text{Send}$ and publish $M = E(m, k)$, E_k , and sta .
- *Receive*(Tid): accept iff $\text{Verify}(vk_{\text{rece}}, \chi, \pi_{\text{rece}}) = 1$, $M_{\text{rece}} = M_{\text{send}}$, $E_{z_B} = E_{z_B}^{\text{ledger}}$, and $\text{sta} = \text{"Send"}$. Set $\text{Tid} \leftarrow \text{Finished}$ on success, else *Invalid* (unique, monotone).

Each R_{Op} coincides with the statement-witness relation of the corresponding circuit.

4) *Lemmas (Circuit & Operation Analyses)*: We keep the essential relations explicit and use them as the R_{Op} checks in $\mathcal{F}_{\text{VCT}^*}$.

Lemma 1 (Deposit/Withdraw Circuit Correctness): For *Deposit/Withdraw*, an accepting proof is valid iff the witness satisfies

$$\text{Deposit: } z^* - z = m, v - v^* = n_c m, m > 0, v^* \geq 0,$$

$$\text{Withdraw: } z - z^* = m, v^* - v = n_c m, m > 0, z^* \geq 0.$$

Sketch: These equalities/inequalities are exactly the statement-witness relations of the circuits. Acceptance implies that these relations hold (soundness), and conversely, if they hold, then a proof exists (completeness). They instantiate R_{dep} and R_{wd} in $\mathcal{F}_{\text{VCT}^*}$. ■

Lemma 2 (Send Operational Correctness): In *Send*, the public inputs include encrypted pre/post zk-balances, $M = E(m, k)$, recipient encapsulation E_k , proof π_{send} , and status flag sta . Any accepting proof enforces, for the sender's zk-balance,

$$z_{\text{snd}}^* = z_{\text{snd}} - m, m > 0, z_{\text{snd}}^* \geq 0,$$

and exposes $\{M, E_k, \text{sta}\}$ for subsequent matching.

Sketch: The *Send* circuit binds the hidden witness to the debit pattern described above, and acceptance attests that these relations hold (soundness). The authenticated public fields are part of the statement and will be matched at *Receive*. This serves as R_{send} for $\mathcal{F}_{\text{VCT}^*}$. ■

Lemma 3 (Receive Correctness and Monotone Consumption): A *Receive* referencing Tid is accepted iff

$$\begin{aligned} &\text{Verify}(vk_{\text{rece}}, \chi, \pi_{\text{rece}}) = 1 \\ &\wedge M_{\text{rece}} = M_{\text{send}} \\ &\wedge E_{z_B} = E_{z_B}^{\text{ledger}} \\ &\wedge \text{sta} = \text{"Send"}. \end{aligned}$$

Upon acceptance, the relay marks Tid as *Finished*, otherwise *Invalid*. Thus, each identifier is consumable at most once.

Sketch: The zk-SNARK check enforces circuit correctness and consistency between the identifier and the ciphertext, and the unique transition *Send*→*Finished/Invalid* yields a single-use monotone state machine. Any second *Receive* either fails these guards or contradicts soundness/consensus. This instantiates R_{rece} and the state invariant used below. ■

5) *Security Definition*: Let $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}^{\text{real}}(1^\lambda)$ be the output of $\mathcal{Z}$ in the real world with Π and $\mathcal{A}$ in the $(\mathcal{G}_{\text{Ledger}}, \mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{XVer}})$ -hybrid model, and

$\text{EXEC}_{\mathcal{F}_{\text{VCT}^*}, \mathcal{F}_{\text{XVer}}, \mathcal{S}, \mathcal{Z}}^{\text{ideal}}(1^\lambda)$ be the output with functionality $\mathcal{F}_{\text{VCT}^*}$ and simulator $\mathcal{S}$.

Definition 1 (UC Realization of $\mathcal{F}_{\text{VCT}^*}$): Protocol Π UC-realizes $\mathcal{F}_{\text{VCT}^*}$ in the $(\mathcal{G}_{\text{Ledger}}, \mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{XVer}})$ -hybrid model if for every PPT adversary $\mathcal{A}$ there exists a PPT simulator $\mathcal{S}$ such that for all PPT environments $\mathcal{Z}$, $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}^{\text{real}} \approx^c \text{EXEC}_{\mathcal{F}_{\text{VCT}^*}, \mathcal{F}_{\text{XVer}}, \mathcal{S}, \mathcal{Z}}^{\text{ideal}}$.

Theorem 1 (Main UC Theorem: Verifiability & Balance Safety): Assuming zk-SNARK completeness and soundness for the four operation circuits and BFT safety for $\mathcal{G}_{\text{Ledger}}$, protocol Π UC-realizes $\mathcal{F}_{\text{VCT}^*}$ in the $(\mathcal{G}_{\text{Ledger}}, \mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{XVer}})$ -hybrid model.

Sketch: A straight-line simulator $\mathcal{S}$ relays $\mathcal{Z}$'s inputs to $\mathcal{F}_{\text{VCT}^*}$ and emulates honest transcripts by running the public verifier and the relay's state machine. If $\mathcal{Z}$ distinguishes the worlds, then either (i) the real relay accepts a transaction that violates some R_{op} although $\mathcal{F}_{\text{VCT}^*}$ rejects, reducing to a zk-SNARK soundness break via Lemma 1 and Lemma 2, or (ii) the same Tid is finalized twice, contradicting the uniqueness/monotonicity invariant of Lemma 3 or the safety of $\mathcal{G}_{\text{Ledger}}$. Hence, the distinguishing advantage is negligible. ■

A detailed UC proof is given in the supplementary material [37].

6) Corollaries:

Corollary 1 (No-Overspending): Under EUF-CMA signatures, binding/hiding commitments, and zk-SNARK soundness, overspending (spending more tokens than they possess) occurs with at most negligible probability.

Sketch: In the ideal world, $\mathcal{F}_{\text{VCT}^*}$ enforces $R_{\text{dep}}, R_{\text{wd}}, R_{\text{send}}$ (Lemma 1, Lemma 2), and these relations preclude negative balances or $\text{out} > \text{in}$. If overspending occurred with non-negligible probability in the real world, $\mathcal{Z}$ would distinguish the two worlds, contradicting Theorem 1. ■

Corollary 2 (No-Double-Spending): Assuming BFT safety for $\mathcal{G}_{\text{Ledger}}$, double-spending [39] occurs with at most negligible probability.

Sketch: In the ideal world, $\mathcal{F}_{\text{VCT}^*}$ consumes each Tid exactly once using the $\text{Send} \rightarrow \text{Finished}/\text{Invalid}$ transition (Lemma 3). A real-world double-spend would let $\mathcal{Z}$ distinguish real from ideal unless it breaks ledger safety or the single-use invariant, contradicting Theorem 1. ■

C. Privacy Analysis

In this subsection, we formalize the privacy goals and threat model and analyze VeCroToken's privacy properties.

1) Scope and Adversary: We analyze privacy against any PPT adversary $\mathcal{A}$ that passively observes all public ledgers (consortium chains and the relay) and controls message delivery in the $(\mathcal{G}_{\text{Ledger}}, \mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{XVer}})$ -hybrid model. The public transcript contains operation types (Deposit/Withdraw/Send/Receive), acceptance bits, zk-SNARK verification keys and proofs, a transfer identifier Tid with a monotone status flag, and ciphertexts such as the amount ciphertext $M = E(m, k)$ and a recipient encapsulation

E_k . No plaintext amounts or plaintext balances are ever published.

2) Leakage Function: Let $\mathcal{L}_{\text{pub}}$ denote the intentional leakage consisting of: (i) operation types and acceptance bits; (ii) Tid and its status maintained by the relay; (iii) public zk-SNARK artifacts (verification keys, proofs); (iv) on-chain account name N and account public key pubk ; and (v) public ciphertexts/headers $(M, E_k, \text{etc.})$, as well as observable transaction metadata (e.g., block timestamps/heights, tx hashes) and lengths of the above objects. We unify the leakage as a function $\mathcal{L}_{\text{pub}}(\text{op}, \text{Tid}, \text{sta}, vk, \pi, (M, E_k, E_z), \text{meta}, \text{len})$. All privacy claims below hold modulo $\mathcal{L}_{\text{pub}}$.

3) Privacy Goals: We formalize three goals aligned with the deployed system.

Definition 2 (Amount Privacy): Fix any ledger schedule and any polynomial-time operation sequence consistent with $\mathcal{L}_{\text{pub}}$. Let $\text{View}^{\text{real}}$ be the joint view of $(\mathcal{A}, \mathcal{Z})$ in the real execution, and let $\text{View}^{\text{ideal}}$ be the view of an ideal functionality that releases only $\mathcal{L}_{\text{pub}}$ while replacing every plaintext amount with a length-matching dummy. Protocol Π achieves *amount privacy* if $\text{View}^{\text{real}} \approx^c \text{View}^{\text{ideal}}$, equivalently, the transaction amount m remains hidden from any party without the decryption key.

Definition 3 (Balance Privacy): Under the same setting, consider an ideal view that publishes $\mathcal{L}_{\text{pub}}$ and replaces the true zk-balances before and after the operation with dummy values of the same length. Protocol Π achieves *balance privacy* if the ideal and real views are computationally indistinguishable, which means that the true balances involved in cross-chain operations are hidden from any party without the relevant keys.

Definition 4 (Fund Correlation Privacy): Let $\mathcal{Q}_0, \mathcal{Q}_1$ be two $\mathcal{L}_{\text{pub}}$ -compatible histories. They induce the same public leakage and the same *multisets* of hidden amounts and hidden balance deltas, but differ in how these hidden values are paired across operations/users/time. Protocol Π achieves *fund correlation privacy* if $\text{View}^{\text{real}}(\mathcal{Q}_0) \approx^c \text{View}^{\text{real}}(\mathcal{Q}_1)$. Intuitively, the public transcript does not reveal the correlation structure between hidden amounts and hidden balance trajectories.

4) Assumptions: We assume IND-CPA [38] security (i.e., semantic security under chosen-plaintext attacks) for the amount encryption and recipient encapsulation, and zero-knowledge for the zk-SNARK (admitting a simulator that produces accepting proofs without witnesses). If balances are encoded via commitments, we assume their hiding property. Public verification depends only on proof acceptance and ciphertext well-formedness, not on plaintexts.

5) Privacy Theorems:

Theorem 2 (Amount Privacy): Under IND-CPA encryption/encapsulation and zk-SNARK zero-knowledge, Π achieves amount privacy.

Sketch: Construct a simulator that (i) uses the zk-SNARK simulator to produce accepting proofs for the public statements and (ii) forms the amount artifacts by encrypting length-matching dummies (for M and E_k) under the prescribed schemes. Public verification only inspects proof acceptance and fixed public fields (including the equality $M_{\text{rece}} =$

M_{send}), which are preserved by construction. If a PPT distinguisher could separate the real transcript from the simulator's output, we obtain either a distinguisher against zk-SNARK zero-knowledge (from the proof objects) or an IND-CPA adversary (from M or E_k). Hence the views are computationally indistinguishable. ■

Theorem 3 (Balance Privacy): Under zk-SNARK zero-knowledge, Π achieves balance privacy.

Sketch: Build a simulator that outputs accepting zk-SNARK proofs via the zero-knowledge simulator and that replaces all balance data (e.g., E_v, E_v^*, E_z, E_z^*) with encryptions of length-matching dummy values under the user's public key. If balances are committed, it replaces the real commitments with hiding-equivalent dummy commitments. Since public checks depend only on proof acceptance and fixed public fields, the simulator preserves acceptance behavior. Any non-negligible distinguisher would yield either a break of zk-SNARK zero-knowledge (from the proof objects) or of IND-CPA/commitment hiding (from the balance artifacts). Therefore, the real and simulated views are indistinguishable. ■

Theorem 4 (Fund Correlation Privacy): Assuming Theorems 2 and 3, Π achieves fund correlation privacy.

Sketch: Let Q_0, Q_1 be $\mathcal{L}_{\text{pub}}$ -compatible histories with identical multisets of hidden amounts and hidden balance deltas but different pairings. Apply the hybrids from Theorem 2 and the simulation from Theorem 3. After these steps, the public view depends on neither concrete amounts nor concrete balance values, only on $\mathcal{L}_{\text{pub}}$, which is identical in Q_0 and Q_1 . Hence, the two distributions are computationally indistinguishable. ■

Remark: Compared with cross-chain transaction linkage-hiding approaches [10], [11], our model hides both amounts and balances by construction and proves the required relations with zk-SNARKs under public verification keys, so the public transcript never reveals plaintext values. It yields formal amount privacy and balance privacy, further achieves fund-correlation privacy, meaning that observers cannot recover how hidden amounts pair with hidden balance trajectories, even when they see the same public leakage. These guarantees do not depend on large concurrent traffic and are not weakened by timing or value side-channel attacks that exploit visible metadata. In contrast, TEE-based interoperability [30], [31] shifts assurance toward enclave operation and operator-maintained logging and attestation, which introduces dependence on specialized hardware and on the availability, integrity, completeness, and impartial access of operator evidence. VeCroToken records publicly verifiable proofs on the relay and consortium ledgers, thereby avoiding those operator-trust risks while preserving auditable correctness.

V. PERFORMANCE EVALUATION

This section presents the experimental evaluation of VeCroToken's zero-knowledge proof overhead and transaction performance. First, it compares the zero-knowledge proof time and space overhead for the four types of transactions: Deposit, Withdraw, Send, and Receive, and contrasts them

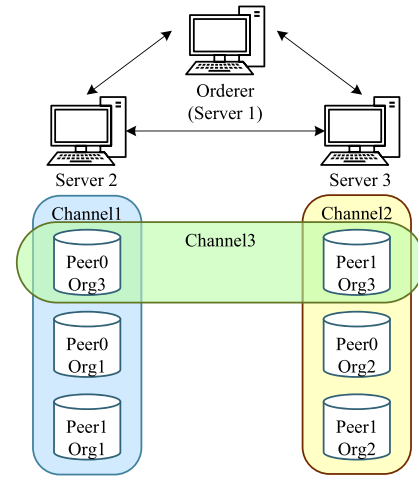


Fig. 6. Our experimental cross-chain network of Hyperledger Fabric consortium blockchains.

TABLE III
THE SERVER CONFIGURATION AND NODE INFORMATION OF THE
CONSORTIUM BLOCKCHAIN CROSS-CHAIN NETWORK

	Server 1	Server 2	Server 3
CPU	Intel Core i9-13900KF	Intel Core i7-11700KF	Intel Core i9-12900KF
Memory	128 GB	48 GB	32 GB
Operating System	Ubuntu 20.04	Ubuntu 22.04	Ubuntu 20.04
Nodes	[orderer. example.com]	[peer0.org1. example.com]	[peer0.org2. example.com]
		[peer1.org1. example.com]	[peer1.org2. example.com]
		[peer0.org3. example.com]	[peer1.org3. example.com]

with the cross-chain transaction models HyperMaze [13] and zkCross [12], which also provide privacy protection and verifiability. Subsequently, this section establishes a consortium blockchain cross-chain network on three servers and evaluates the transaction performance of the model, analyzing the relationship between the processing rate and processing latency of the four transaction operations with varying transaction request rates. Furthermore, the section assesses the impact of the number of servers on the transaction performance of VeCroToken.

A. Experimental Setup

We utilize the jsnark [17] library to construct zero-knowledge proof circuits for various transaction operations in VeCroToken and implement the complete workflow of zk-SNARKs using the libsark [40] library, where all transaction operations' zk-SNARKs schemes are based on the Groth16 protocol [18]. We also build a consortium blockchain cross-chain network based on the Hyperledger Fabric 2.4 framework and combine it with the zero-knowledge proof library to complete the operation process of all transactions. Hyperledger Fabric is a permissioned blockchain platform with a modular architecture and consensus with deterministic finality. It has seen broad adoption in industry [41] and academia [13], [31].

As illustrated in Fig. 6, we deploy a cross-chain network of Hyperledger Fabric consortium blockchains consisting of three servers. Table III describes the server configurations and node information. One server with superior configuration is designated as the ordering node, while the remaining two servers each host a separate channel, representing two independent consortium blockchains. It is worth noting that in Hyperledger Fabric, each channel can be considered an independent consortium blockchain. These channels are isolated from each other, each maintaining its ledger. Three peer nodes are deployed in each channel (Channel 1 and Channel 2), with one peer node belonging to a different organization in each channel. These specific nodes serve as relay nodes. In addition to their original channels, all relay nodes join another channel (Channel 3), acting as the relay chain. In Hyperledger Fabric, to facilitate cross-channel (cross-chain) information exchange, nodes must simultaneously join these two channels and use a specific method, `InvokeChaincode`, to call chaincode across channels. Each relay node has a chaincode installed on the two channels it has joined. Thanks to the support for cross-channel invocation of chaincode provided by the `InvokeChaincode` method, these nodes can call consortium chaincode from the relay chaincode, thereby accessing consortium blockchain ledgers across channels.

Consequently, relay nodes can query relevant data from consortium blockchain ledgers and store it in the relay chain's ledger. Similarly, relay nodes can call relay chaincode from consortium chaincode, retrieving relevant data from the relay chain's ledger and storing it in consortium blockchain ledgers. As the number of consortium blockchains increases, such deployment will support more nodes joining the relay chain, facilitating interconnection among more consortium blockchains.

B. Evaluation of Zero-Knowledge Proof Overhead

Space consumption. Table IV compares zero-knowledge proof space overhead for each transaction operation. As shown, the Deposit and Withdraw transactions exhibit remarkably similar sizes in terms of the pk and vk parameters, mainly due to the high degree of similarity in circuit design between these two types of transactions. The same pattern also applies to transactions between Send and Receive and between Send* and Receive*. Moreover, since Deposit and Withdraw transactions have the highest circuit complexity, the sizes of their pk and vk parameters are naturally the largest. On the other hand, the space overhead of Send* and Receive* transactions also reaches approximately 1.5 times that of Send and Receive transactions. Additionally, the size of each transaction's proof π is 134 B, indicating that the size of proof π is only influenced by the zk-SNARKs scheme.

Time consumption. Fig. 7 compares zero-knowledge proof (zk-SNARKs) time overhead for various transaction operations. To demonstrate the performance improvement brought by employing the dual-balance model to separate the token conversion process in cross-chain transfer transactions, we also conduct circuit construction and overhead tests specifically for Send and Receive transactions involving token conversion

TABLE IV
ZK-SNARKS SPACE CONSUMPTION OF EACH TRANSACTION OPERATION

	Pk-size	Vk-size	Proof-size
Deposit	116.17MB	6.00KB	134B
Withdraw	116.17MB	6.00KB	134B
Send	66.01MB	4.41KB	134B
Receive	65.98MB	4.41KB	134B
Send*	94.65MB	5.82KB	134B
Receive*	94.62MB	5.82KB	134B

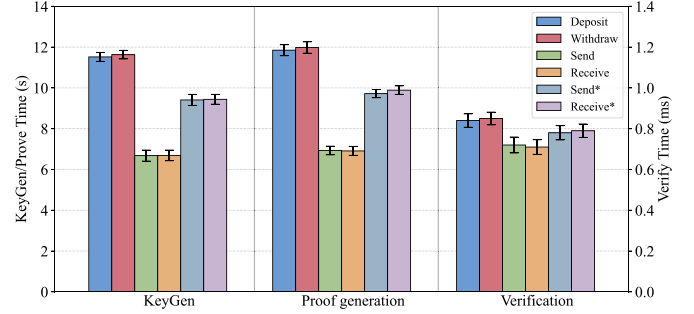


Fig. 7. The time efficiency of each transaction operation (KeyGen and Proof generation correspond to the left ordinate, while Verification corresponds to the right ordinate).

(labeled as Send* and Receive* in the subsequent figures and tables, respectively). As shown in the figure, the time overhead for key pair generation, proof generation, and proof verification stages of Deposit and Withdraw transactions is significantly higher than other types of transactions. A comparison of the circuit constructions for Deposit/Withdraw transactions and transfer transactions reveals that, apart from the commitment relations, Deposit and Withdraw transactions each involve four asymmetric encryption, whereas Send and Receive transactions involve only two asymmetric encryption and one symmetric encryption. It suggests that asymmetric encryption consumes more time than symmetric encryption and commitment relations. Furthermore, compared to transfer transactions without token conversion, transfer transactions with token conversion only add one asymmetric encryption in circuit construction, but their time overhead for key pair and proof generation increases by nearly 3 seconds, further confirming that asymmetric encryption constitutes a significant portion of the overall time consumption.

Comparison with other methods in time consumption. We implement HyperMaze [13] and zkCross [12], and measure the time consumption of their zero-knowledge transactions (HyperMaze: Convert, Redeem, Send, Receive; zkCross: Redeem, Mint) in the same experimental environment as VeCroToken. Fig. 8 illustrates the time consumption of these methods and our VeCroToken. Note that the zero-knowledge proof generation time measurement in HyperMaze, zkCross, and VeCroToken includes encoding format conversion, data type unification, and data serialization. Additionally, in zkCross the Burn and Transmit stages introduce some computational overhead (e.g., hashing and Merkle-path construction), but they do not invoke any zero-knowledge prover

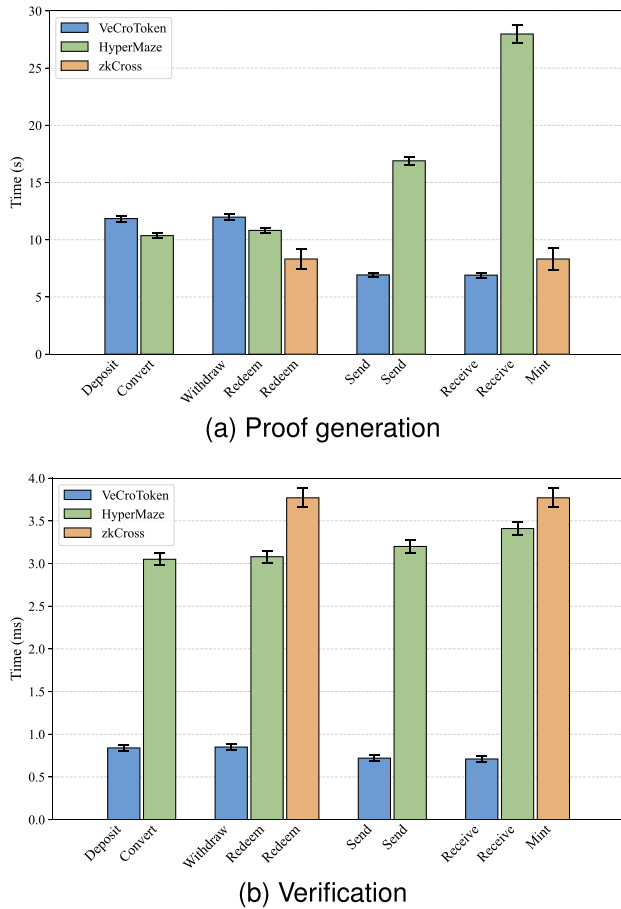


Fig. 8. Comparison with other methods in time consumption.

or verifier, so we exclude these stages from the ZK-related time comparison.

As can be seen from Fig. 8, for transactions related to token conversion, the proof generation time for the Deposit operation and Withdraw operation in VeCroToken is approximately 11.85 s and 11.98 s, slightly higher than the Convert operation and Redeem operation in HyperMaze, which are 10.36 s and 10.82 s respectively. The Redeem proof generation time of zkCross is 8.32 s, lower than that of VeCroToken and HyperMaze. HyperMaze adopts a dual-balance model consisting of plaintext and zk-balances, primarily used to protect the privacy of balances rather than for token conversion in cross-chain transactions. Thus, HyperMaze encrypts only the zk-balance before and after the transaction, whereas VeCroToken encrypts both balances before and after. By contrast, zkCross hides receiver addresses and transfer amounts, employs independent preimages for hash locks, thereby providing transaction unlinkability guarantees but not encrypting balances. For cross-chain transfer transactions, the expenditure transaction operation Send and the income transaction operation Receive in VeCroToken both require approximately 6.9 s, and the Mint in zkCross also requires 8.32 s, while the time-consuming Send and Receive operations in HyperMaze reach unacceptable durations of 16.90 s and 27.98 s, respectively, which is very time-consuming for a single transfer transaction. Regarding

proof verification, the proof verification time for all transaction operations in VeCroToken is approximately 0.7 to 0.8 ms, less than one-third of that in HyperMaze and zkCross.

Remarks. Although VeCroToken's proof generation time for Deposit and Withdraw transactions is slightly higher than that for the corresponding transactions in HyperMaze, the impact of these two types of transactions on overall performance is limited due to their relatively small proportion in practical application scenarios. However, for crucial cross-chain transfer transactions, VeCroToken demonstrates significant advantages, better aligning with the demands of real-world applications. The identical Redeem and Mint times of zkCross stem from the shared circuit/CRS and nearly identical constraint systems. Notably, unlike VeCroToken and HyperMaze, zkCross's proof generation time grows approximately linearly with the block size (i.e., the number of transactions per block). Additionally, VeCroToken outperforms HyperMaze and zkCross in verification time for each transaction operation. HyperMaze's slower Send/Receive operations stem from its monolithic circuit design, which combines balance encryption and token conversion. In zkCross, the verifier must carry more public inputs (e.g., block/state roots and serial numbers) as part of the statement. It increases the constant factor of Groth16 verification and thus leads to a longer verification time than VeCroToken. In contrast, VeCroToken decouples these via a dual-balance mechanism, reducing circuit complexity. Conversely, HyperMaze's simpler Convert/Redeem operations omit dual-balance checks, making it slightly faster in those phases.

C. Evaluation of Transaction Performance

This section evaluates the transaction performance of consortium blockchains enhanced with our VeCroToken. We utilize a performance testing tool called Caliper [42] specifically designed for Hyperledger Fabric to conduct tests on the consortium blockchain cross-chain network constructed in this study. It continuously sends requests to the Fabric network through the Node SDK and measures the performance of the Fabric network based on the responses received, including throughput (TPS) and latency (ms). We simulate the behavior of nodes continuously generating four types of cross-chain transaction requests and writing relevant information into the ledger. The request rate is set to gradually increase from 150 to 900 TPS, enabling a comprehensive evaluation of VeCroToken's throughput (throughput = total committed/total time) and latency (confirmation time - submission time).

We also evaluate the impact of the number of servers on the transaction performance of consortium blockchains enhanced with our VeCroToken. We gradually increase the number of servers from initially 1 to eventually 5. The configurations of servers 1, 2, and 3 are detailed in Table III, while the configurations of servers 4 and 5 are described in Table V. Note that when the number of servers does not exceed 2, although the number of consortium blockchains remains at 2, these chains are deployed on the same server. However, when the number of servers exceeds 3, each additional server corresponds to an additional consortium blockchain. Moreover,

TABLE V
THE ADDED SERVER CONFIGURATIONS AND NODE INFORMATION

	Server 4	Server 5
CPU	Intel Core i9-12900KS	Intel Core i9-14900K
Memory	64 GB	128 GB
Operating System	Ubuntu 22.04	Ubuntu 20.04
Nodes	peer0.org4.example.com	peer0.org5.example.com
	peer1.org4.example.com	peer1.org5.example.com
	peer3.org3.example.com	peer4.org3.example.com

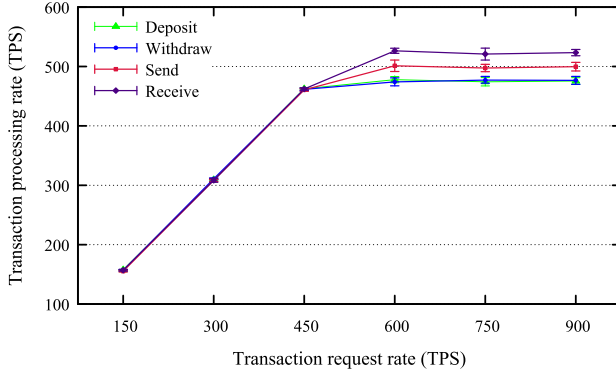


Fig. 9. The transaction processing rate of VeCroToken.

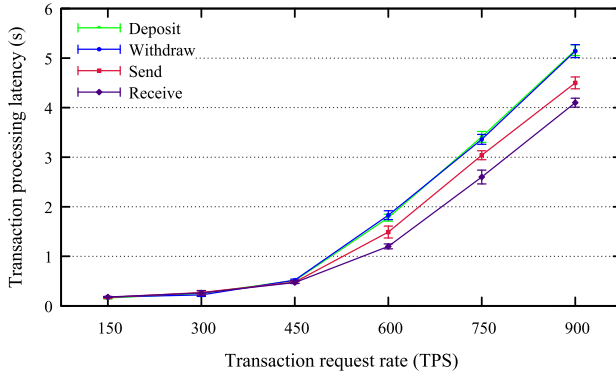


Fig. 10. The transaction processing latency of VeCroToken.

regardless of the variation in the number of servers, the ordering node is always deployed on server 1.

Transaction processing rate. Fig. 9 illustrates the relationship between the processing rate of the four types of transactions in VeCroToken and the request rate. As illustrated, when the request rate does not exceed 450 TPS, the processing rates of the four types of transactions show synchronous growth, with minimal differences among them. During this stage, all submitted transactions can be processed immediately. However, when the request rate exceeds 450 TPS, noticeable differentiation in the processing rates of different transactions begins to emerge and gradually stabilizes, indicating that the consortium blockchain cross-chain network is approaching its performance bottleneck. The processing rates of Deposit and Withdraw transactions remain very close due to their highly similar public data requirements. Meanwhile, Receive transactions requiring the least amount of on-chain data exhibit the highest processing rate among all transaction types. Ultimately, the processing rates of Deposit, Withdraw, Send, and Receive transactions stabilize at 476 TPS, 477 TPS, 500 TPS, and 520 TPS, respectively.

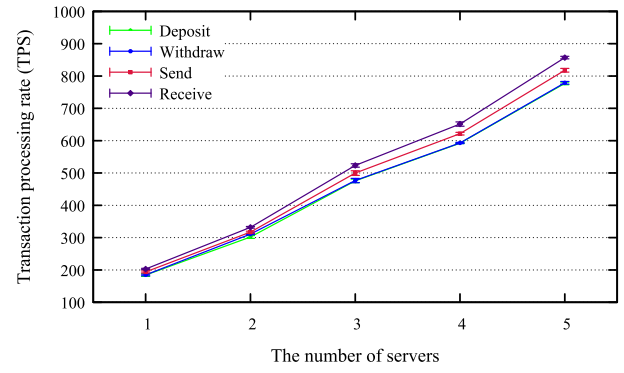


Fig. 11. The relationship between VeCroToken's transaction processing rate and the number of servers.

Transaction processing latency. Fig. 10 illustrates the relationship between the processing latency of the four types of transactions in VeCroToken and the request rate. As illustrated, when the request rate is below 450 TPS, the processing latency of all transactions is very short, with none exceeding 0.5 seconds. However, when the request rate exceeds 450 TPS, the processing latency of each transaction operation begins to increase rapidly. Specifically, at a request rate of 600 TPS, the processing latency of all transactions ranges from 1 to 2 seconds. After reaching 900 TPS, the processing latency of all transactions exceeds 4 seconds, with the processing latency of Deposit and Withdraw transactions exceeding 5 seconds. In the performance tests conducted in this section, we only send transactions of varying rates to the consortium blockchain cross-chain network for 10 seconds. If the consortium blockchain cross-chain network has yet to reach its performance bottleneck, the duration of sending transaction requests has little impact on the processing latency of transactions. However, the consortium blockchain cross-chain network cannot process the transaction requests promptly once reaching its performance bottleneck. Continuously sending more transaction requests to VeCroToken will result in an increasing backlog of unprocessed transaction requests, increasing the average transaction processing latency. Therefore, it is essential to avoid running VeCroToken for extended periods under its performance bottleneck in practical applications to ensure efficient and stable system operation.

The impact of the number of servers on the transaction performance. Fig. 11 shows the relationship between VeCroToken's transaction processing rates and the number of servers. As shown, the processing rates of the four types of transactions significantly increase as the number of servers increases, demonstrating a steady upward trend. Additionally, performance differences between these transactions become more apparent as transaction processing rates increase. Furthermore, the increase in transaction processing rates is closely related to the performance of the added servers. It can be seen that the transaction processing rates increase the most when the number of servers increases from 4 to 5, since the performance of our five servers varies, and server 5 has the best performance.

Remarks. VeCroToken demonstrates exceptional performance. When the consortium blockchain cross-chain network is not at its load limit, the processing rate of each transaction operation can reach up to approximately 500 TPS, with processing delays not exceeding 0.5 seconds. In contrast, HyperMaze achieves only tens to hundreds of TPS per chain, with transaction processing delays of close to 10 seconds. Moreover, the overall performance of the entire consortium blockchain cross-chain network enhanced with VeCroToken is also significantly improved as the number of deployed servers and consortium chains increases. Due to condition limitations, we use a relatively small number of servers with varying performance. Suppose VeCroToken were to be deployed on more servers. In that case, the transaction performance should be improved and stabilized until the ordering node approaches its performance bottleneck.

VI. DISCUSSIONS

In this section, we focus on our model's practical aspects and possible extensions. We discuss heterogeneous cross-chain scenarios, the trusted setup and updatability of zk-keys, regulatory auditability with verifiable logs and clear provenance, and expanding the scope of privacy preservation to richer metadata while maintaining public verification and efficient execution.

Researching heterogeneous cross-chain scenarios. VeCroToken is primarily based on the Hyperledger Fabric platform and within homogeneous cross-chains. The security mechanisms, consensus algorithms, network topologies, and block generation verification logic are consistent among homogeneous chains, making cross-chain interactions relatively straightforward. However, cross-chain interactions among heterogeneous chains are more complex. For example, Bitcoin uses the PoW algorithm, while the consortium chain Fabric uses a traditional deterministic consensus algorithm. The composition and deterministic guarantee mechanisms of their blocks are significantly different. Further research is needed to address heterogeneous cross-chain scenarios, which may require the assistance of third-party services for cross-chain interactions.

Trusted setup and updatability of zk-keys. Our zero-knowledge proof workflow follows the standard interface $\Pi = (\text{Setup}, \text{GenKey}, \text{Prove}, \text{Verify})$. In real deployments, *Setup* should be run via an MPC ceremony (at least one honest participant) with transcripts and randomness-beacon sources published. It could clarify the operational trust boundary of the public parameters and mitigate the potential risk of a setup ceremony compromise. Any circuit change triggers a fresh *GenKey* and a new *vk*. Proofs under an old *vk* need not verify under the new one, while previously proved statements remain valid. In production, rotations could be managed by a governance-controlled public registry that records versioned verification keys and provenance, binds each *vk* to a hash of the circuit/constraint system, announces an activation time/height with a short grace window accepting both old and new keys, and supports emergency revocation. It also makes the admissible key set auditable over time. Trusted setup hygiene and

a lightweight, governance-backed key-rotation policy could provide a concrete path to robust, real-world deployments.

Regulatory auditability. VeCroToken supports regulator-facing, ex post audits. The relay chain verifies zero-knowledge proofs using public parameters. It records the verification outcomes together with the transaction data for third-party auditing. Send/Receive records expose only the necessary public fields (e.g., account names, transaction identifiers, statuses, and the posted public statements/credentials). An auditor, acting as a read-only third party, reproduces relay-node checks by invoking *TxVerify* and, when applicable for cross-chain transfers, validating the paired Send/Receive with the same public verification keys, and reports the outcome off-chain. No plaintext balances or amounts are disclosed. Although amounts remain hidden, the observable fields permit auditors to assess transfer direction, construct account-level and pairwise activity counts, which support identifying anomalous interaction patterns relevant to anti-money laundering oversight under consortium governance policies. As future work, we will focus on privacy-preserving regulatory audit methods that rely only on public statements and verification keys.

Expanding the scope of privacy preservation. To meet regulatory review and public verifiability, VeCroToken protects balances, transfer amounts, and their relations privacy, while exposing only ciphertexts, public statements, account names, and verification keys so auditors can validate correctness without plaintext. It satisfies basic compliance needs but leaves residual linkability across transactions. To extend privacy protection from values to identities and behavioral patterns, we could combine receiver stealth addressing with a one-time sender treatment, without altering the external verification model. For each transfer, the sender generates fresh ephemeral key material and derives a unique one-time destination using the receiver's view and spend key. Only the ephemeral element and the one-time destination appear on the chain, and no long-term recipient identifier is revealed, which prevents clustering of multiple receipts for the same account. The amount remains symmetrically encrypted, but the key is derived independently by both endpoints via key agreement, removing the need to publish a separate key field. The receiver wallet scans the ledger using only the view key, reconstructs the shared secret from the posted ephemeral element, attempts decryption of the amount, and checks that the derived destination matches the posted destination. If successful, it records a local spendable entry. It will introduce additional endpoint computation and scanning overhead, but maintain public auditability and reduce linkability.

VII. CONCLUSION

This work presents a verifiable and privacy-preserving cross-chain model for consortium blockchains based on zk-SNARKs, referred to as VeCroToken. VeCroToken adopts a dual-balance mechanism, enabling token conversion between consortium blockchains to improve the efficiency of cross-chain transfer transactions. The encrypted data of cross-chain

transactions and generated zero-knowledge proof credentials are stored in the ledgers of the consortium blockchain, where the transaction parties are located, and the relay chain. It allows relay nodes and third parties in other consortium chains to verify the credentials of various transactions using relevant parameters and public data, protecting the privacy of cross-chain transactions while ensuring their legitimacy and verifiability. We provide a detailed exposition of the specific structure and transaction data information of VeCroToken, introduce its model construction, and analyze its security and privacy. We deploy a cross-chain network of Hyperledger Fabric consortium blockchains and implement our VeCroToken, combining the jsnark and libsark libraries for comprehensively evaluating VeCroToken's overhead and performance. The experimental results show that VeCroToken has a lower overall zero-knowledge proof overhead than the state-of-the-art models and performs well in transaction performance. In the future, we will focus on developing a verifiable, privacy-preserving cross-chain model for heterogeneous blockchain platforms, enhancing regulatory auditability and extending privacy preservation.

REFERENCES

- [1] E. Androulaki et al., "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proc. EuroSys Conf. (EuroSys)*, 2018, pp. 1–15.
- [2] "Hyperledger: Advancing business blockchain adoption through global open source collaboration," Accessed: Jan. 16, 2024. [Online]. Available: <https://www.hyperledger.org>
- [3] "Corda: An introduction," R3. Accessed: Jan. 16, 2024. [Online]. Available: <https://www.r3.com/>
- [4] "Corda: The future of digital finance is built on trust," R3. Accessed: Jan. 16, 2024. [Online]. Available: <https://www.corda.net>
- [5] X. Zhang, S. Jiang, Y. Liu, T. Jiang, and Y. Zhou, "Privacy-preserving scheme with account-mapping and noise-adding for energy trading based on consortium blockchain," *IEEE Trans. Netw. Services Manag.*, vol. 19, no. 1, pp. 569–581, Mar. 2022.
- [6] M. M. Islam, M. K. Islam, M. Shahjalal, M. Z. Chowdhury, and Y. M. Jang, "A low-cost cross-border payment system based on auditable cryptocurrency with consortium blockchain: Joint digital currency," *IEEE Trans. Services Comput.*, vol. 16, no. 3, pp. 1616–1629, May/Jun. 2023.
- [7] L. Xue, H. Huang, F. Xiao, and W. Wang, "A cross-domain authentication scheme based on cooperative blockchains functioning with revocation for medical consortiums," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 2409–2420, Sep. 2022.
- [8] Q. Feng, D. He, S. Zeadally, M. K. Khan, and N. Kumar, "A survey on privacy protection in blockchain system," *J. Netw. Comput. Appl.*, vol. 126, pp. 45–58, Jan. 2019.
- [9] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 1999, pp. 173–186.
- [10] J. Cai, Y. Zhou, T. Hu, and B. Li, "PTLC: Protect the identity privacy during cross-chain asset transaction more effectively," in *Proc. IEEE Int. Conf. Softw. Qual. Rel. Secur. Companion (QRS-C)*, 2022, pp. 70–78.
- [11] L. Hanzlik, J. L. Sri, A. Thyagarajan, and B. Wagner, "Sweep-UC: Swapping coins privately," in *Proc. IEEE Symp. Security Privacy (SP)*, 2024, pp. 3822–3839.
- [12] Y. Guo et al., "zkCross: A novel architecture for cross-chain privacy-preserving auditing," in *Proc. USENIX Secur. Symp. (USENIX Security)*, Philadelphia, PA, Aug. 2024, pp. 6219–6235. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/guo-yihao>
- [13] W. Liu, Z. Wan, J. Shao, and Y. Yu, "Hypermaze: Towards privacy-preserving and scalable permissioned blockchain," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 360–376, Jan./Feb. 2021.
- [14] J. Groth, "On the size of pairing-based non-interactive arguments," in *Proc. Adv. Cryptol.*, 2016, pp. 305–326.
- [15] J. Groth and M. Maller, "Snarky signatures: Minimal signatures of knowledge from simulation-extractable SNARKs," in *Proc. Adv. Cryptol.*, 2017, pp. 581–612.
- [16] B. Parno, J. Howell, C. Gentry, and M. Raykova, "Pinocchio: Nearly practical verifiable computation," *Commun. ACM*, vol. 59, no. 2, pp. 103–112, Jan. 2016.
- [17] K. Ahmed, "jsnark: A java library for zk-SNARK circuits," Accessed: Jan. 16, 2024. [Online]. Available: <https://github.com/akosba/jsnark>
- [18] J. Groth, "Short pairing-based non-interactive zero-knowledge arguments," in *Proc. Adv. Cryptol.*, 2010, pp. 321–340.
- [19] E. B. Sasson et al., "Zerocash: Decentralized anonymous payments from bitcoin," in *Proc. IEEE Symp. Secur. Priv. (SP)*, 2014, pp. 459–474.
- [20] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *Proc. IEEE Symp. Secur. Priv. (SP)*, 2018, pp. 315–334.
- [21] "Agzs/BlockMaze," Accessed: Jan. 16, 2024. [Online]. Available: <https://github.com/agzs/BlockMaze>
- [22] X. Hu, W. Zhou, J. Yin, G. Cheng, S. Yan, and H. Wu, "Towards verifiable and privacy-preserving account model on a consortium blockchain based on zk-SNARKs," *Peer-to-Peer Netw. Appl.*, vol. 16, no. 4, pp. 1834–1851, May 2023.
- [23] R. Belchior, A. Vasconcelos, S. Guerreiro, and M. Correia, "A survey on blockchain interoperability: Past, present, and future trends," *ACM Comput. Surv.*, vol. 54, no. 8, p. 168, Oct. 2021.
- [24] G. Wang, Q. Wang, and S. Chen, "Exploring blockchains interoperability: A systematic survey," *ACM Comput. Surv.*, vol. 55, no. 13S, p. 290, Jul. 2023.
- [25] A. Augusto, R. Belchior, M. Correia, A. Vasconcelos, L. Zhang, and T. Hardjono, "SoK: Security and privacy of blockchain interoperability," in *Proc. IEEE Symp. Security Privacy (SP)*, 2024, pp. 3840–3865.
- [26] "Wanchain's cross-chain solution," Wanchain. Accessed: Sep. 5, 2024. [Online]. Available: <https://www.wanchain.org/>
- [27] Harmony Team, "Harmony one-ETH bridge," Accessed: Sep. 5, 2024. [Online]. Available: <https://bridge.harmony.one/>
- [28] "Chainlink cross-chain interoperability protocol (CCIP)," Chainlink Labs. Accessed: Sep. 5, 2024. [Online]. Available: <https://chain.link/cross-chain>
- [29] "Introducing the chainlink platform privacy suite: CCIP private transactions, blockchain privacy manager, and deco sandbox," Accessed: Sep. 5, 2024. [Online]. Available: <https://blog.chain.link/ccip-private-transactions-blockchain-privacy-manager/>
- [30] M. Li et al., "IvyCross: A privacy-preserving and concurrency control framework for blockchain interoperability," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 9334–9351, Oct. 2025.
- [31] R. Belchior, A. Vasconcelos, M. Correia, and T. Hardjono, "Hermes: Fault-tolerant middleware for blockchain interoperability," *Future Gener. Comput. Syst.*, vol. 129, pp. 236–251, Apr. 2022.
- [32] S. Goldwasser, S. Micali, and C. Rackoff, *The Knowledge Complexity of Interactive Proof-Systems*. New York, NY, USA: Assoc. Comput. Mach., 2019, pp. 203–225.
- [33] M. Blum, P. Feldman, and S. Micali, "Non-interactive zero-knowledge and its applications," in *Proc. ACM Symp. Theory Comput. (STOC)*, 1988, pp. 103–112.
- [34] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, "From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again," in *Proc. Innov. Theor. Comput. Sci. Conf. (ITCS)*, 2012, pp. 326–349.
- [35] T. Chen, H. Lu, T. Kunpittaya, and A. Luo, "A review of zk-SNARKs," Feb. 2022, *arXiv:2202.06877*.
- [36] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proc. IEEE Symp. Found. Comput. Sci. (FOCS)*, 2001, pp. 136–145.
- [37] K. Ding, X. Hu, W. Zhou, G. Cheng, R. Li, and H. Wu, "VecroToken: An efficient, verifiable, and privacy-preserving cross-chain model for consortium blockchains based on zk-SNARKs (supplementary material)," Accessed: Nov. 15, 2025. [Online]. Available: https://github.com/acacams/VeCroToken/blob/main/VeCroToken_Supplementary_Material.pdf
- [38] J. Katz and Y. Lindell, *Introduction to Modern Cryptography: Principles and Protocols*. Boca Raton, FL, USA: Chapman and Hall/CRC, 2007.
- [39] U. W. Chohan, "The double spending problem and cryptocurrencies," Dec. 2021. [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3090174
- [40] V. Madars et al., "LibSNARK: A c++ library for zkSNARK proofs," Accessed: Jan. 16, 2024. [Online]. Available: <https://github.com/scipr-lab/libsnark>

- [41] "IBM blockchain platform," Accessed: Sep. 5, 2024. [Online]. Available: <https://www.ibm.com/think/topics/hyperledger>
- [42] "A blockchain benchmark framework to measure performance of multiple blockchain solutions," LF Decentralized Trust, Accessed: Jan. 16, 2024. [Online]. Available: <https://www.hyperledger.org/use/caliper>



Ke Ding (Graduate Student Member, IEEE) received the M.S. degree in software engineering from East China Normal University in 2023. He is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Southeast University, Nanjing, China. His research interests include blockchain security and network security.



Xiaoyan Hu (Member, IEEE) received the Ph.D. degree in computer architecture from Southeast University, Nanjing, China, in 2015. She visited the NetSec Laboratory, Colorado State University from 2010 to 2012. She is currently an Associate Professor with the School of Cyber Science and Engineering, Southeast University. Her research interests include network intrusion detection, network security, and future network architecture.



Weicheng Zhou received the B.S. degree from the Wuhan University of Technology, Wuhan, China, in 2021, and the M.S. degree from Southeast University, Nanjing, China, in 2024. His research interests include zero-knowledge proof and blockchain privacy protection.



Guang Cheng (Member, IEEE) received the Ph.D. degree in computer science from Southeast University, Nanjing, China, in 2003. From 2006 to 2007, he was a Postdoctoral Fellow with the School of Electrical and Computer Engineering, Georgia Institute of Technology. He is currently a Full Professor with the School of Cyber Science and Engineering, Southeast University. His research interests include network security, network measurement, and encrypted traffic analysis.



Ruidong Li (Senior Member, IEEE) received the B.S. degree in engineering from Zhejiang University, Hangzhou, China, in 2001, and the M.S. and Ph.D. degrees from the University of Tsukuba, Japan, in 2005 and 2008, respectively. He was a Researcher with the Network Architecture Laboratory, National Institute of Information and Communications Technology. He is currently an Associate Professor with the Institute of Science and Engineering, Kanazawa University. His current research interests include Internet of Things, future networks, information-centric networking, security/secure architectures of future networks, and next-generation wireless networking. He serves on the editorial board for IEEE INTERNET OF THINGS JOURNAL and *KSII Transactions on Internet and Information Systems*.



Hua Wu (Member, IEEE) received the Ph.D. degree in computer application technology from Southeast University, Nanjing, China, in 2010, where she is currently an Associate Professor with the School of Cyber Science and Engineering. Her research interests include computer network security and encrypted traffic analysis.